

INGENIERÍA CIVIL MATEMÁTICA

Documento Explicativo: Implementación de Métodos Numéricos para EDPs

Nombre :
Cristóbal Alexis Reyes Pavez

1. Contenido:

Este documento describe el desarrollo e implementación de diversos métodos numéricos para resolver ecuaciones diferenciales parciales (EDP) en Python. Los códigos presentados están diseñados para abordar problemas lineales homogéneos y no homogéneos, con condiciones de borde tipo Robin.

2. Primera Problemática:

Paquetes

Para operar los códigos presentados en este documento se necesitan utilizar los siguientes paquetes

- **numpy**: Operaciones matemáticas y construcción de matrices.
- **numpy.linalg**: Contiene funciones específicas para álgebra lineal. **eigvals** Para calcular valores propios. **norm** Calcula normas de vectores.
- **scipy**: Funciones especializadas. **expm** Exponenciales matriciales. **legendre** Genera polinomios de Legendre.
- **pandas** Gestión y análisis numérico de tablas.
- **time** Se utiliza para medir el tiempo de ejecución de diferentes bloques de código.

Códigos en Python

El método presentado genera el operador diferencial asociado a una ecuación diferencial parcial homogénea con condiciones de borde tipo Robin y funciones de coeficientes dependientes de x . El operador es de la forma:

$$a_2(x) \frac{\partial^2 u}{\partial x^2} + a_1(x) \frac{\partial u}{\partial x} + a_0(x) u = f(x) \quad (1)$$

La implementación calcula las matrices de discretización para los términos de segunda y primera derivada, aplicando condiciones de borde tipo Robin en ambos extremos del dominio.

Operador de segunda derivada

```
1 def matriz_op_seg_derivada(n):  
2     matrix = np.zeros((n, n))  
3     np.fill_diagonal(matrix, -2)  
4     np.fill_diagonal(matrix[1:], 1)  
5     np.fill_diagonal(matrix[:, 1:], 1)  
6     return matrix
```

Operador de primera derivada centrada

```
1 def matriz_op_prim_derivada_centrada(n):
2     matrix = np.zeros((n, n))
3     np.fill_diagonal(matrix[1:], -1)
4     np.fill_diagonal(matrix[:, 1:], 1)
5     return matrix
```

La función `edp.lineal` discretiza ecuaciones de la forma (1) entregando la matriz L representando el operador diferencial discretizado y el vector no homogéneo asociado a las condiciones de borde.

Operador lineal de EDP

```
1 def edp_lineal(a2_func, a1_func, a0_func, f_no_homo, coef_borde_ini,
2               coef_borde_fin, n, L):
3     ##### Inicializacion #####
4     A1, B1, C1 = coef_borde_ini
5     A2, B2, C2 = coef_borde_fin
6     dx = L / (n - 1)
7     xx = np.linspace(0, L, n)
8     x = xx[1:n-1]
9
10    #####
11    #Escritura del aporte del aporte de las condiciones de borde al operador
12    #diferencial
13    #####
14    e_ini = 1 / (dx * A1 - B1)
15    e_fin = 1 / (dx * A2 + B2)
16
17    #####
18    #Construccion del operador de segunda derivada con condiciones de borde
19    #####
20    d2 = 1 / (dx ** 2) * (matriz_op_seg_derivada(n - 2) +
21                          np.diag([-B1*e_ini if i == 0 else B2*e_fin if i == n - 3 else 0
22                                for i in range(n - 2)]))
23
24    #####
25    #Construccion del operador de primera derivada con condiciones de borde
26    #####
27    d1 = 1 / (2*dx) * (matriz_op_prim_derivada_centrada(n - 2) +
28                      np.diag([B1*e_ini if i == 0 else B2*e_fin if i == n - 3 else 0
29                              for i in range(n - 2)]))
30
31    #####
32    #Construccion del aportes de los coeficientes
33    #####
34    a2_matrix = np.diag([a2_func(xi) for xi in xx[1:n-1]])
35    a1_matrix = np.diag([a1_func(xi) for xi in xx[1:n-1]])
36    a0_matrix = np.diag([a0_func(xi) for xi in xx[1:n-1]])
37
38    #####
39    #Construccion del operador diferencial agregando los coeficientes
40    #####
41    L = a2_matrix @ d2 + a1_matrix @ d1 + a0_matrix
42
43    #####
```

```

43     #Construccion del termino no homogeneo con el aporte de las condciones de
        borde
44     #####
45     no_homogeneo = 1 / (dx) * np.array(C1*[e_ini] + [0] * (n - 2) +
46         C2*[e_fin]) + np.array(C1*[-e_ini] + [0] * (n - 2) +
47         C2*[e_fin]) - np.array([f_no_homo(xi) for xi in xx[1:n-1]])
48     return L, no_homogeneo

```

De acuerdo a las necesidades del problema 1 la función euler_eplicito_dt implementa el método de Euler explícito para resolver sistemas de ecuaciones diferenciales representante al problema transitorio de la forma

$$\frac{d\vec{u}}{dt} = A\vec{u}$$

con un paso temporal dt. En este caso, el método devuelve la matriz de transición, sus valores propios y su norma, lo cual permite analizar la estabilidad del sistema en función de dt.

```

1 def euler_explicito_dt(A, dt):
2     n = A.shape[0]
3     Id = np.identity(n)
4     T = Id + dt * A
5     T_eigvals = np.linalg.eigvals(T)
6     norm_T = np.linalg.norm(T, ord=2)
7     return T, T_eigvals, norm_T

```

Se es innecesario resolver el problema estacionario ya que este no tiene termino no homogéneo.

Código Análisis Numérico

```

1 ##### Parametros #####
2 def f_no_homo(x):
3     return 0
4 def a1_func(x):
5     return 0
6
7 def a0_func(x):
8     return 0
9
10 def u0(x):
11     return np.sin(x*np.pi)
12
13 def solucion_analitica(x, t):
14     return np.sin(x*np.pi)*np.exp(-t*np.pi**2)
15
16
17 coef_borde_ini= [1 , 0 , 0]
18 coef_borde_fin= [1 , 0 , 0]
19 L = 1
20
21 T_max = 10
22 L = 1
23 N_vals = [20, 100, 200]
24 M_vals = [100, 1000, 10000]
25 D_vals=[-1,0.01,1,100]
26
27 resultados = []
28
29 #Bucle sobre los diferentes valores del coeficiente de difusion D
30 for param in D_vals:

```

```

31 a2_func = lambda x: param
32 for n in N_vals:
33     dx = L / (n - 1)
34     x_vals = np.linspace(0, L, n - 2)
35
36     for m in M_vals:
37         dt = T_max / m
38         Tiempos = np.arange(0, m * dt, dt)
39
40
41     # Construccion de la matriz del sistema para la EDP
42     A, no_homogeneo = edp_lineal(a2_func, a1_func, a0_func, f_no_homo,
43                                   coef_borde_ini, coef_borde_fin, n, L)
44
45     u_exacta_final = solucion_analitica(x_vals, Tiempos[-1])
46
47     # Propagacion temporal con el metodo de Euler explicito
48     T, T_eigvals, norm_T = euler_explicito_dt(A, dt)
49
50     U0 = u0(x_vals)
51
52     u_aprox = U0 @ np.linalg.matrix_power(T, m)
53
54     max_eigenvalue = max(abs(T_eigvals))
55     Estabilidad = max_eigenvalue <= 1
56
57     error = np.linalg.norm(u_exacta_final - u_aprox)
58
59     # Registro de los resultados
60     resultados.append({
61         'D': param,
62         'D*dt/dx^2': param*dt/(dx**2),
63         'Norma T': norm_T,
64         'Estabilidad': 'Estable' if Estabilidad else # Agrupar los resultados
65             'Inestable',
66         'Error': error
67     })
68
69     tabla_resultados = pd.DataFrame(resultados)
70
71     # Ordenar la tabla de resultados por el parametro D*dt/dx^2
72     tabla_resultados_ordenada = tabla_resultados.sort_values('D*dt/dx^2')
73
74     # Agrupar los resultados por valor de D
75     for D_vals, tabla_dx in tabla_resultados_ordenada.groupby('D'):
76         print(f"                Resultados para D = {D_vals}")
77         print(tabla_dx)
78         print("\n" + "-"*40 + "\n")

```

Problemática 2:

Códigos en Python

Trapesio Simple

Este código implementa el método del trapezio simple para la integral integración numérica de $f(x)$ definida en el intervalo $[a, b]$ mediante:

$$\int_a^b f(x)dx \approx \frac{b-a}{2}[f(a) + f(b)]$$

```
1 def trapesio_simple(f, a, b):
2     return (b - a) * (f(a) + f(b)) / 2
```

Aquí los factores importantes son

- **f** Función a integrar
- **a, b** Limite inferior y superior del intervalo de integración

Cuya **salida** es el calculo aproximado del área bajo la curva.

Cuadratura Gaussiana

El código siguiente calcula los nodos (raíces) y los pesos basándose en los polinomios de Legendre. Por ultimo se realiza la aproximación de la integral usando los nodos y pesos obtenidos del forma:

$$\int_a^b f(x)dx \approx \frac{b-a}{2} \sum_{i=1}^n f\left(\frac{b-a}{2}x_i + \frac{a+b}{2}\right)$$

Calculo nodos y pesos

```
1 def gauss_legendre_pesos(n):
2     raices = np.roots(legendre(n))
3     pesos = []
4     for i in range(n):
5         peso = 2 / ((1 - raices[i]**2) * (legendre(n).deriv(1)(raices[i]))**2)
6         pesos.append(peso)
7
8     return pesos, raices
```

Aquí los factores importantes son

- **n** Orden de la cuadratura, que determina el número de nodos y pesos.
- **np.roots(legendre(n))** función especializada en calcula las raíces del polinomio de Legendre de grado n , que son los nodos.

La **salida** son las raices y pesos del método para el orden n .

Aplicación de la cuadratura

```
1 def cuadratura_gaussiana(f, a, b, pesos, nodos):
2     integral = 0
3     N=len(pesos)
4     for i in range(N):
5         xi = ((b - a)*(nodos[i]+1))/2 + a
6         integral += pesos[i] * f(xi)
7
8     return ((b - a)/2)*integral
```

Los factores importantes aquí son:

- **f** Función a integrar
- **a,b** Limites de integración
- **pesos** y **nodos** Obtenidos operando el código anterior

Recalcar que la función viene implementada para transformar el intervalo $[a, b]$ al intervalo $[-1, 1]$ para aplicar la cuadratura.

Método De Galerkin

El método de Galerkin implementado aproxima numéricamente el resultado de una EDP elípticas lineales de segundo orden con coeficientes variables y con termino no homogéneo considerando condiciones de borde homogéneas con su forma de divergencia presente, es decir de la forma:

$$-\frac{\partial}{\partial x}(p(x)\frac{\partial u}{\partial x}) + q(x)u = f(x)$$

En este caso, se implementa con dos esquemas de integración: trapecio simple y Gauss-Legendre.

Galerkin con integración trapezoidal

```

1 def galerkin(f, p, q, g_inicial, N, L, dt, M):
2     #####Inicializacion#####
3     dx = L / (N - 1)
4     xx = np.linspace(0, L, N)
5     m=0
6     c=np.zeros(N)
7     num_cond[]
8     #####Iteracion sobre el tiempo#####
9
10    while m <= M:
11        if m == 0:
12            #Condicion inicial t=0
13            c = np.array([g_inicial(xx)])
14            m = m + 1
15        else:
16            #####
17            #Construccion matriz correspondiente al termino bilineal bilineal
18            #####
19
20            A = np.zeros((N-2, N-2))
21
22            #Funcion que se integra en la diagonal
23            integral_iA= lambda x, x_ii: p(x)+(1/dt + q(x))*(x-x_ii)**2
24
25            #Funcion que se integra fuera de la diagonal
26            integral_iB= lambda x, x_ii, x_i: p(x)+(1/dt + q(x))*(x-x_ii)*(x-x_i)
27
28            diag_valores = (1/dx**2) * (trapesio_simple(lambda x: integral_iA(x, xx
29                [0:-2]), xx[0:-2], xx[1:-1]) +
30                trapesio_simple(lambda x: integral_iA(x, xx[2:]),
31                    , xx[1:-1], xx[2:]))
32
33            diag_sup_valores= -(1/dx**2) * (trapesio_simple(lambda x: integral_iB(x, xx
34                [2:], xx[1:-1]), xx[1:-1], xx[2:]))
35
36            #Almacenaje de datos
37            A[0:, 0:] = np.diag(diag_valores)

```

```

35     np.fill_diagonal(A[1:], diag_sup_valores)
36     np.fill_diagonal(A[:,1:], diag_sup_valores)
37
38     #####
39     #Construccion vector correspondiente al termino de la derecha
40     #####
41
42
43     F = np.zeros(N-2)
44
45     #Funcion que se integra en la diagonal (notar la influencia de la
46     #discretizacion temporal en el termino fuente con c[m.1]/dt.
47     integral_iC = lambda x, x_ii, i: (f(x) + c[m-1][i] / dt) * (x - x_ii)
48
49     F_valores = (1 / dx) * (trapesio_simple(lambda x: integral_iC(x, xx[0:-2],
50     np.arange(1, len(xx)-1)), xx[0:-2], xx[1:-1]) -
51     trapesio_simple(lambda x: integral_iC(x, xx[2:], np.
52     arange(1, len(xx)-1)), xx[1:-1], xx[2:]))
53
54     # Almacenaje de datos
55     F[0:] = F_valores
56
57     #####
58     #Resolucion sistema lineal Ac=F
59     c_new = np.linalg.solve(A,F)
60
61     #Contribuciones homogeneas de los bordes
62     c0 = 0
63     c1 = 0
64
65     c_new=np.insert(c_new,0,c0)
66     c_new=np.insert(c_new,N-1,c1)
67
68     #Almacenaje de resultado para el tiempo m
69     c=np.vstack([c, c_new])
70
71     # Calcular y almacenar el numero de condicionamiento de la matriz A para
72     # el tiempo m
73     num_cond.append(np.linalg.cond(A))
74     m = m + 1
75
76     return c,num_cond

```

Aquí las variables clave son:

- **f** Fuente en la ecuación diferencial.
- **p** y **q** Funciones coeficientes
- **g_inicial** Condición inicial en $t = 0$.
- **N** numero de nodos espaciales.
- **L** longitud del intervalo
- **dt** tamaño pasos temporales
- **M** Número de pasos temporales.

Este código entrega como **salida** 2 arreglos de datos. El primero con los coeficientes c_i^m para cada paso temporal m y en segundo lugar el numero de condicionamiento de la matriz A para cada tiempo m .

Galerkin con integración Gauss-Legendre

El esquema es similar al anterior, pero utiliza la cuadratura de Gauss-Legendre.

```
1 def galerkin_gauss_legendre(f, p, q, g_inicial, N, a, b, dt, M, Nc):
2     #####Inicializacion#####
3
4     dx = (b-a) / (N - 1)
5     xx = np.linspace(a, b, N)
6     m = 0
7     pesos, raices = gauss_legendre_pesos(Nc)
8     c=np.zeros(N)
9     num_cond=[]
10
11     #####Iteracion sobre el tiempo#####
12
13     while m <= M:
14         if m == 0:
15             #Condicion inicial t=0
16             c = np.array([g_inicial(xx)])
17             m = m + 1
18         else:
19             #####
20             #Construccion matriz correspondiente al termino bilineal
21             #####
22
23             A = np.zeros((N-2, N-2))
24
25             #Funcion que se integra en la diagonal
26             integral_iA= lambda x, x_ii: p(x)+(1/dt + q(x))*(x-x_ii)**2
27
28             #Funcion que se integra fuera de la diagonal
29             integral_iB= lambda x, x_ii, x_i: p(x)+(1/dt + q(x))*(x-x_ii)*(x-x_i)
30
31             diag_valores = (1/dx**2) * (cuadratura_gaussiana(lambda x: integral_iA(x, xx
32                 [0:-2]), xx[0:-2], xx[1:-1], pesos, raices) +
33                 cuadratura_gaussiana(lambda x: integral_iA(x, xx
34                 [2:]), xx[1:-1], xx[2:], pesos, raices))
35
36             diag_sup_valores= -(1/dx**2) * (cuadratura_gaussiana(lambda x: integral_iB(x
37                 ,xx[2:],xx[1:-1]), xx[1:-1], xx[2:], pesos, raices))
38
39             #Alacenaje de datos
40             A[0:, 0:] = np.diag(diag_valores)
41             np.fill_diagonal(A[1:], diag_sup_valores)
42             np.fill_diagonal(A[:,1:], diag_sup_valores)
43
44             # #####
45             #####
46             #Construccion vector correspondiente al termino de la derecha
47             #####
48
49             F = np.zeros(N-2)
50
51             # Funcion que se integra en la diagonal ( notar la influencia de la
52                 discretizacion temporal en el termino fuente con c [ m .1]/ dt
53             integral_iC = lambda x, x_ii, i: (f(x) + c[m-1][i] / dt) * (x - x_ii)
```



```

53     F_valores = (1/dx)* (cuadratura_gaussiana(lambda x: integral_iC(x, xx[0:-2],
54         np.arange(1, len(xx)-1)), xx[0:-2], xx[1:-1], pesos, raices) -
        cuadratura_gaussiana(lambda x: integral_iC(x, xx
            [2:], np.arange(1, len(xx)-1)), xx[1:-1], xx[2:],
            pesos, raices))
55
56     # Almacenaje de dato
57     F[0:] = F_valores
58
59     # #####
60
61     #Resolucion sistema lineal Ac=F
62     c_new = np.linalg.solve(A,F)
63
64     # Contribuciones homogeneas de los bordes
65     c0 = 0
66     c1 = 0
67
68     c_new=np.insert(c_new,0,c0)
69     c_new=np.insert(c_new,N-1,c1)
70
71     # Almacenaje de resultado para el tiempo m
72     c=np.vstack([c, c_new])
73
74     # Calcular y almacenar el numero de condicionamiento de la matriz $A$ para
        el tiempo m
75     num_cond.append(np.linalg.cond(A))
76     m = m + 1
77     return c,num_cond

```

Las principales diferencias son:

- **Nc** Se agrega esta entrada para definir el orden de la cuadratura que se quiere utilizar
- **pesos** y **nodos** Se calculan una vez al comienzo del codigo
- **cuadratura_gaussiana** Se llama este codigo en lugar de **trapesio_simple**

Código Análisis Numérico

```

1 def solucion_analitica(x, t):
2     return np.sin(x * np.pi) * np.exp(-t * np.pi**2)
3
4 def calcular_error(sol_num, sol_analitica):
5     return np.linalg.norm(sol_num - sol_analitica)
6
7 # Parametros
8 a, b = 0, 1
9 g = lambda x: np.sin(np.pi * x)
10 f = lambda x: 0
11 p = lambda x: 1
12 q = lambda x: 0
13
14 # Valores de dx a probar
15 dx_vals = [0.1, 0.05, 0.01, 0.005]
16 dt_vals = [0.1, 0.05, 0.01, 0.005]
17
18 for dx in dx_vals:
19     # Calcular N basado en el dx dado

```

```

20 N = int(1 / dx) + 1
21
22 resultados = []
23
24 for dt in dt_vals:
25     M = int(1 / dx) + 1
26     xx = np.linspace(a, b, N)
27     t_max = dt * M
28     tiempos = np.linspace(0, t_max, M + 1)
29     sol_analitica_vals = np.array([solucion_analitica(xx, t) for t in tiempos
30                                     ])
31
32     # Resolver con ambos metodos
33     c_gauss_legendre = galerkin_gauss_legendre(f, p, q, g, N, a, b, dt, M, 6)
34     c_galerkin = galerkin(f, p, q, g, N, b - a, dt, M)
35
36     # Calcular errores
37     error_gauss_legendre = calcular_error(c_gauss_legendre[0],
38                                           sol_analitica_vals)
39     error_galerkin = calcular_error(c_galerkin[0], sol_analitica_vals)
40
41     # Tomar el mayor numero de condicionamiento en el tiempo
42     condicion_gauss_legendre = np.max(c_gauss_legendre[1]) if len(
43         c_gauss_legendre[1])>0 else 0
44
45     condicion_galerkin = np.max(c_galerkin[1]) if len(c_galerkin[1])>0 else 0
46
47     resultados.append({
48         "dt": dt,
49         "Error Gauss-Legendre": error_gauss_legendre,
50         "Error Trapesoidal": error_galerkin,
51         "Condicionamiento Gauss-Legendre": condicion_gauss_legendre,
52         "Condicionamiento Trapesoidal": condicion_galerkin
53     })
54
55     # Convertir resultados a DataFrame y mostrar la tabla
56     df_resultados = pd.DataFrame(resultados)
57     print(f"Tabla para dx = {dx}")
58     print(df_resultados.to_string(index=False))
59     print("\n" + "-"*40 + "\n")

```

3. Problemática 3:

Codigos en Python

Este código define un sistema de ecuaciones diferenciales parciales (EDP) de segundo orden con la siguiente forma.

$$a_5(x) \frac{\partial^2 u}{\partial x^2} + a_4(x) \frac{\partial^2 u}{\partial x \partial y} + a_3(x) \frac{\partial^2 u}{\partial y^2} + a_2(x) \frac{\partial u}{\partial x} + a_1(x) \frac{\partial u}{\partial y} + a_0(x) u = f(x, y)$$

En esta formulación y el código no se consideran **las condiciones de borde por lo que se asumen homogéneas**.

Submatrices

Las funciones `matrix_central`, `matrix_superior` y `matrix_inferior` calculan las formas matriciales de los bloques de la matriz principal. Es decir se calcula respectivamente C , S e I para la matriz A que es de la forma

$$A = \text{Tridiag}_{(M-2) \times (N-2)}(S, C, I)$$

Cada una de estas funciones toma como entradas:

- **a5_func, a4_func, a3_func, a2_func, a1_func, a0_func** Correspondiente a las funciones coeficientes de la formulación principal
- **xi** e **yi** correspondiente a las coordenadas discretizadas del espacio.
- **n** Tamaño del mallado en x interna sin incluir bordes
- **dx** y **dy** tamaño de los pasos

Cada función retorna una matriz representa un rasgo del operador diferencial.

```
1 def matriz_central(a5_func, a4_func, a2_func, a0_func, xi, yi, n, dx, dy):
2     matrix = np.zeros((n, n))
3     np.fill_diagonal(matrix, -2*( a5_func(xi,yi)/dx**2 + a4_func(xi,yi)/dy**2 ) +
4         a0_func(xi,yi))
5     np.fill_diagonal(matrix[1:], a5_func(xi,yi)/dx**2 + a2_func(xi,yi)/(2*dx))
6     np.fill_diagonal(matrix[:, 1:], a5_func(xi,yi)/dx**2 + a2_func(xi,yi)/(2*dx))
7     return matrix
8
9 def matriz_superior(a4_func, a3_func, a1_func, xi, yi, n, dx, dy):
10    matrix = np.zeros((n, n))
11    np.fill_diagonal(matrix, (1/dy)*( a4_func(xi,yi)/dy + a1_func(xi,yi)/2 ))
12    np.fill_diagonal(matrix[1:], a3_func(xi,yi)/(4*dx*dy))
13    np.fill_diagonal(matrix[:,1:], -a3_func(xi,yi)/(4*dx*dy))
14    return matrix
15
16 def matriz_inferior(a4_func, a3_func, a1_func, xi, yi, n, dx, dy):
17    matrix = np.zeros((n, n))
18    np.fill_diagonal(matrix, (1/dy)*( a4_func(xi,yi)/dy - a1_func(xi,yi)/2 ))
19    np.fill_diagonal(matrix[1:], -a3_func(xi,yi)/(4*dx*dy))
20    np.fill_diagonal(matrix[:,1:], a3_func(xi,yi)/(4*dx*dy))
21    return matrix
```

Operador diferencial Dos Dimensional

La función `edp_lineal_orden_2` discretiza el dominio espacial en una malla de tamaño $N \times M$ y construye la matriz que representa el acutar discretizado del operador diferencial en dos dimensiones.

```
1
2 def edp_lineal_orden_2( f, a5_func, a4_func, a3_func, a2_func, a1_func, a0_func ,
3     N, M, Ln, Lm):
4     #####Inicializacion Problema#####
5     dx = Ln / (N - 1)
6     dy = Lm / (M - 1)
7     xi = np.linspace(0, Ln, N)
8     yi = np.linspace(0, Lm, M)
9
10    x = xi[1:N-1]
11    y = yi[1:M-1]
12
```

```

13 n=N-2
14 m=M-2
15
16 #####
17 #Contruccion Submatrices
18 #####
19
20
21 C = matriz_central(a5_func, a4_func, a2_func, a0_func, x, y, n, dx, dy)
22 S = matriz_superior(a4_func, a3_func, a1_func, x, y, n, dx, dy)
23 I = matriz_inferior(a4_func, a3_func, a1_func, x, y, n, dx, dy)
24
25 #####
26 #Ensamblaje matriz principal
27 #####
28
29 matrix = np.zeros((n*m, n*m))
30 for i in range(m):
31     # Matriz central
32
33     matrix[ (n * i): (n * (i + 1)), (n * i):(n * (i + 1))] = C
34
35     #Ensamblar matriz inferior con el condicional i>0 e i<m-1 para no iterar en
    vacio:
36
37     if i > 0:
38         matrix[(n * i):(n * (i + 1)), (n * (i - 1)):(n * i)] = I
39     if i < m - 1:
40         matrix[(n * i):(n * (i + 1)), (n * (i + 1)):(n * (i + 2))] = S
41
42     #####
43     #Ensamblaje matriz de terminos fuente
44     #####
45
46 F=np.array([f(i, j) for i in x for j in y])
47
48 return matrix,F

```

La función `edp_lineal_orden_2` discretiza el dominio espacial en una malla de tamaño $N \times M$ y construye la matriz que representa el actuar discretizado del operador diferencial en dos dimensiones asumiendo un formato de organización por columnas.

Esta aplicación tiene algunas particularidades. Se discretizan las dimensiones del dominio en los pasos $\Delta x = \frac{L_n}{N-1}$ y $\Delta y = \frac{L_m}{M-1}$. No se consideran las condiciones de borde por lo que esto queda para un trabajo posterior

Código Análisis Numérico

Esta función realiza las simulaciones para diferentes tamaños de malla $N = M$, evaluando el error y el tiempo de ejecución:

```

1 def calculo_tabla(f, exact_sol, a5_func, a4_func, a3_func, a2_func, a1_func,
2   a0_func, tamanos):
3     resultados = []
4
5     for N in tamanos:
6         M = N
7         dx = 1 / (N - 1)
8         dy = 1 / (M - 1)
9
10        # Tiempo de inicio

```

```

10     Tiempo_inicio = time.time()
11
12     # Construir matriz A y vector F
13     A, F = edp_lineal_orden_2(f, a5_func, a4_func, a3_func, a2_func, a1_func,
14                               a0_func, N, M, 1, 1)
15
16     # Resolver el sistema
17     u_num = np.linalg.solve(A, F)
18
19     # Tiempo de fin
20     Tiempo_final = time.time() - Tiempo_inicio
21
22     # Generar malla para calcular la solucion exacta
23     x = np.linspace(0, 1, N)
24     y = np.linspace(0, 1, M)
25     x, y = np.meshgrid(x[1:-1], y[1:-1]) # Excluir los bordes
26     u_exact = exact_sol(x, y).flatten()
27
28     # Calcular normas
29     l2_norm = np.sqrt(np.sum((u_exact - u_num)**2) * dx * dy)
30     linf_norm = np.max(np.abs(u_exact - u_num))
31
32     # Guardar resultados
33     resultados.append({
34         "N x M": f"{N} x {M}",
35         "dx, dy": dx,
36         "L2 Norm": l2_norm,
37         "Linf Norm": linf_norm,
38         "Tiempo de simulacion (s)": Tiempo_fin
39     })
40
41     return resultados
42
43 def u_exact(x, y):
44     return np.sin(np.pi * x) * np.sin(np.pi * y)
45
46 def a5_func(x,y):
47     return -1
48
49 def a4_func(x,y):
50     return -1
51
52 def a3_func(x,y):
53     return 0
54
55 def a2_func(x,y):
56     return 0
57
58 def a1_func(x,y):
59     return 0
60
61 def a0_func(x,y):
62     return 0
63
64 def f(x, y):
65     return (2 * np.pi**2) * np.sin(np.pi * x) * np.sin(np.pi * y)
66
67 Tamanos = [10, 20, 40, 60, 80, 120, 140, 160]

```

```

68
69 resultados = calculo_tabla(f, u_exact, a5_func, a4_func, a3_func, a2_func, a1_func
    , a0_func, Tamanos)
70
71
72 df = pd.DataFrame(resultados)
73 print(df)
74
75 df.to_csv("results.csv", index=False)
76
77 N_values = [int(value.split(" x ")[0]) for value in df["N x M"]]
78 times = df["Tiempo de ejecucion (s)"]
79
80
81 plt.figure(figsize=(10, 6))
82 plt.plot(N_values, times, marker='o')
83 plt.xlabel("N")
84 plt.ylabel("Tiempo de ejecucion (s)")
85 plt.title("N vs. Tiempo de ejecucion")
86 plt.grid(True)
87 plt.show()

```